



SEMESTER 2 EXAMINATIONS 2023/2024

MODULE: CA216 - Operating Systems

PROGRAMME(S):

COMSCI	BSc in Computer Science
ECSAO	Study Abroad (Engineering & Computing)
CASE	BSc in Computer Applications (Sft.Eng.)

YEAR OF STUDY: 2,0

EXAMINER(S):

Dr. Graham Healy	(Internal)	(Ext:7364)
Prof. Arend Rensink	(External)	External

TIME ALLOWED: 2 Hours

INSTRUCTIONS: Answer 4 questions. All questions carry equal marks.

PLEASE DO NOT TURN OVER THIS PAGE UNTIL YOU ARE INSTRUCTED TO DO SO.

The use of programmable or text storing calculators is expressly forbidden.

Please note that where a candidate answers more than the required number of questions, the examiner will mark all questions attempted and then select the highest scoring ones.

There are no additional requirements for this paper.

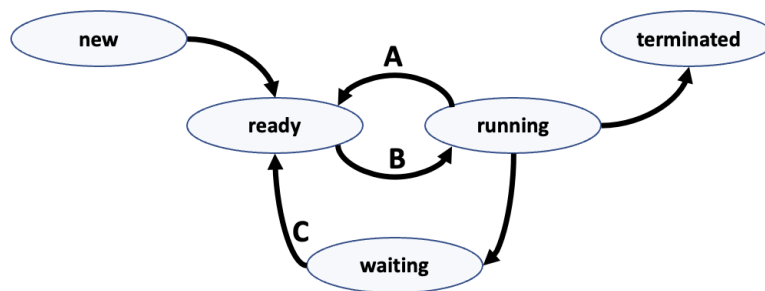
QUESTION 1

[TOTAL MARKS: 25]

Q 1(a)

[6 Marks]

Consider the process life-cycle figure shown below, where processes change between various states during their lifetime.



Describe why each of the following state transitions of a process might occur:

- i) transition from a *running* state to a *ready* state (A)
- ii) transition from a *ready* state to a *running* state (B)
- iii) transition from a *waiting* state to a *ready* state (C)

Q 1(b)

[11 Marks]

Consider the following C program that will be compiled and executed on Linux.

```
01: #include <stdio.h>
02: #include <unistd.h>
03: #include <sys/types.h>
04: #include <sys/wait.h>

05: int main()
06: {
07:     pid_t pid;

08:     pid = fork();

09:     if (pid < 0) {
10:         return 1;
11:     }
12:     else if (pid == 0) {
13:         execlp("/bin/ls", "ls", NULL);
14:         printf("Here123 \n");
15:     }
16:     else {
17:         wait(NULL);
18:     }

19:     return 0;
}
```

- i) Explain what the *fork()* and *execlp()* calls do.
- ii) Will the print statement on line 14 be executed? Explain why or why not.
- iii) What is the purpose of the call to *wait()* on line 17?

Q 1(c)**[4 Marks]**

What information would we expect to find about a process in a Process Control Block about?

Q 1(d)**[4 Marks]**

The *size* command on Linux gives us information about the size of memory for initialised data, uninitialised data, and code size. Why doesn't the *size* command on Linux give us information about the heap size of a program?

[End of Question 1]

QUESTION 2**[TOTAL MARKS: 25]****Q 2(a)****[5 Marks]**

What is Amdahl's law? What does Amdahl's law tell us about a program?

Q 2(b)**[6 Marks]**

Shared memory and *message passing* are two ways that processes on a system can communicate. Explain what both of these are.

Q 2(c)**[9 Marks]**

Consider the following C program that will be compiled and executed on Linux. The writer of the code expected an output of 200,000 to be printed, however, the value 177,249 was printed. With reference to specific line numbers, explain the problem and describe what changes you would make to the code to resolve it.

```
01: #include <stdio.h>
02: #include <threads.h>
03: #include <stdatomic.h>

04: int cnt;

05: int f(void* thr_data)
06: {
07:     for(int n = 0; n < 10000; ++n)
08:         ++cnt;

09:     return 0;
10: }

11: int main(void)
12: {
13:     thr_t thr[20];
14:     for(int n = 0; n < 20; ++n)
15:         thr_create(&thr[n], f, NULL);
16:     for(int n = 0; n < 20; ++n)
17:         thr_join(thr[n], NULL);
18:     printf("The counter is %u\n", cnt);
19: }
```

Q 2(d)**[5 Marks]**

Is the following statement true or false? Explain why or why not?

Using a round robin scheduler in an operating system will mean that some processes are never scheduled to execute.

[End of Question 2]

QUESTION 3**[TOTAL MARKS: 25]****Q 3(a)****[11 Marks]**

Explain what is meant by the term **deadlock prevention**. In your answer, describe each of the 4 necessary conditions for deadlock to occur.

Q 3(b)**[8 Marks]**

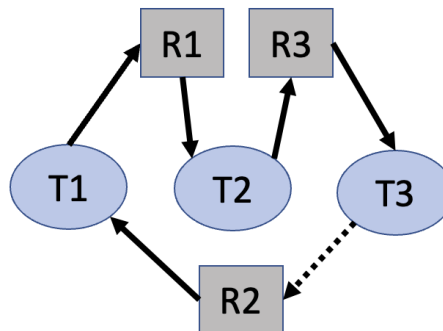
Consider the following Python code for the dining philosophers problem. Will this code result in deadlock? Explain why or why not. In your answer, make reference to specific line numbers.

<pre> 01: import threading, random, time 02: def philosopher_func(name, left, right): 03: while True: 04: fork1, fork2 = left, right 05: print(name, "thinking") 06: time.sleep(random.uniform(1,10)) 07: while True: 08: fork1.acquire(True) 09: locked = fork2.acquire(False) 10: if locked: 11: break 12: fork1.release() 13: fork1, fork2 = fork2, fork1 14: print(name, "dining") 15: time.sleep(random.uniform(1,10)) 16: fork2.release() 17: fork1.release() </pre>	<pre> 18: forks = [threading.Lock() for n in range(5)] 19: names = ('Aristotle','Kant','Spinoza','Marx', 'Russel') 20: for i in range(5): 21: t = threading.Thread(target=philosopher_func, 22: \ 23: args=(names[i], forks[i%5], 24: forks[(i+1)%5])) 25: t.start() </pre>
---	---

Q 3(c)**[6 Marks]**

Consider the resource-allocation graph below for a system where a single instance is available for each resource (i.e. R1, R2, and R3). T1/T2/T3 are tasks/processes. Is this system in a safe, unsafe or deadlocked state? Explain your answer.

Note: a broken-arrow line indicates a claim edge.



[End of Question 3]

QUESTION 4**[TOTAL MARKS: 25]****Q 4(a)****[7 Marks]**

What does the term *internal fragmentation* mean in the context of memory management? Use an example as part of your answer.

Q 4(b)**[6 Marks]**

In the context of a filesystem, briefly explain what an *inode* is.

Q 4(c)**[6 Marks]**

In paging memory management, *pages* may be shared between multiple processes. Explain a situation where this is beneficial.

Q 4(d)**[6 Marks]**

Explain what is meant by the term DMA (Direct Memory Access). In your answer, explain why DMA would be used instead of programmed I/O.

[End of Question 4]

[END OF EXAM]